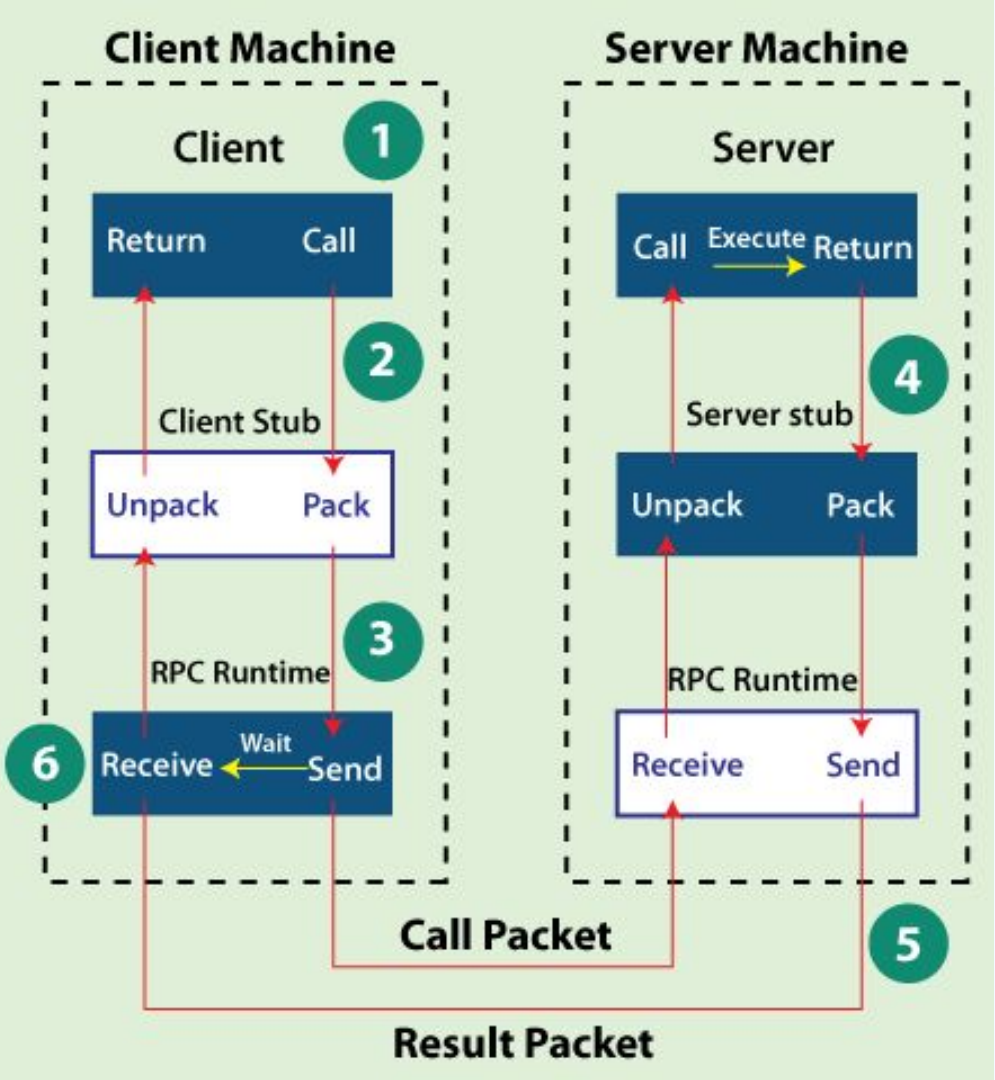
SISTEMAS DISTRIBUÍDOS

Comunicação

Prof Priscila Marques Kai

RCP


Client Machine
Server Machine
Client
1
Return
Call
2
Client Stub
Unpack
Pack
3
RPC Runtime
6
Receive
Wait
Send
Server
Call
Execute
Return
4
Server stub
Unpack
Pack
RPC Runtime
Receive
Send
Call Packet
5
Result Packet

Client stub:

Função: obtêm os parâmetros de uma função (ex: append(data, dbList))

- Empacota tudo em uma **mensagem** (transforma em bytes),
- E envia para o **server stub**.

parameter marshaling

(serialização)

Client stub:

Função: obtêm os parâmetros de uma função (ex: append(data, dbList))

- Empacota tudo em uma **mensagem** (transforma em bytes),
- E envia para o **server stub**.
- Do outro lado, o servidor faz o *unmarshaling* (desserialização - desempacota e reconstrói os dados na memória).

No entanto...

- A rede **só transmite bytes** (sequência de 0 e 1).
  - os bytes **não carregam metadados automáticos** dizendo o tipo e que informação guarda
  - enviar "bytes crus" não informam ao servidor **como interpretar o que recebeu**.

Por isso:

- É preciso **descrever os parâmetros** (metainformação).
- Exemplo: o servidor precisa saber que os primeiros 4 bytes representam o tamanho da lista, depois vem os elementos, etc.

RCP – SUPORTE A APLICAÇÕES BASEADAS EM RPC

Para que um RPC funcione, **cliente e servidor precisam concordar** em como as mensagens são formatadas e interpretadas.

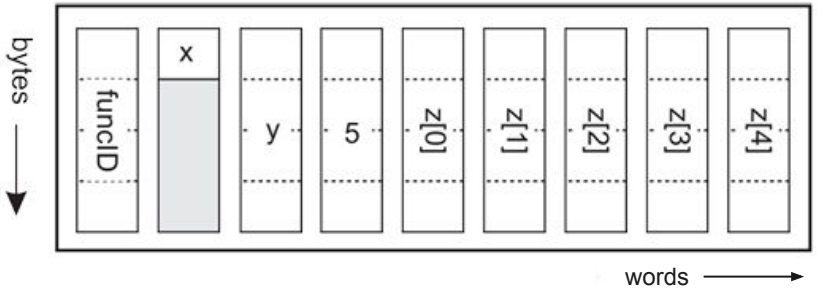
- Inclui:
  - ordem dos parâmetros,
  - codificação de tipos (inteiros, floats, arrays, strings etc.)

Formatação de mensagens

Considere a função

```
void someFunction(char x; float y; int z[5])
```

(a)



(b)

Figure 4.11: (a) A function. (b) The corresponding message, and the order in which bytes and words are sent across the network.

Formatação de mensagens

Considere a função

```
someFunction(char x; float y; int z[5]).
```

- O protocolo pode definir:
  - `char` ocupa o último byte de uma *palavra*
  - `float` ocupa uma *palavra inteira*.
  - `array` vem precedido pelo tamanho, seguido dos elementos.

unidade de dados de tamanho fixo em bits

O **stub do cliente** segue esse formato para empacotar os parâmetros, enquanto o **stub do servidor** usa o mesmo formato para interpretar os dados recebidos.

Transporte da mensagem

- Após definir o formato e a codificação, é preciso acordar o mecanismo de transporte:
  - Pode ser **TCP** (orientado a conexão) ou
  - **UDP** (datagramas com controle de erros implementado na aplicação).

RPC síncrono

Exemplo prático:

Função remota **append**

```
new_list = append([1, 2, 3])
```

Abra o arquivo RPC.py e confira o exemplo de procedimento remoto.

RPC síncrono

Exemplo prático:

Função remota `append`

```
new_list = append([1, 2, 3])
```

Uso de Sockets

Muitos sistemas distribuídos usam diretamente o modelo de mensagens do nível de transporte.

- É a camada mais básica da comunicação entre programas pela rede, usando protocolos como:
  - TCP → comunicação confiável, em ordem
  - UDP → comunicação mais simples e rápida, mas sem garantia de entrega

RPC síncrono

Exemplo prático:

Função remota `append`

```
new_list = append([1, 2, 3])
```

Uso de Sockets

Ou seja, em vez de trabalhar com algo mais “alto nível” (como RPC, gRPC, REST), eles trocam mensagens cruas usando TCP ou UDP por meio do uso de sockets.

RPC síncrono

Exemplo prático:

Função remota **append**

```
new_list = append([1, 2, 3])
```

Operações de socket para TCP

No lado do servidor, geralmente a sequência é:

- socket() → cria o endpoint de comunicação.
- bind() → associa o socket a um endereço (IP + porta).
- listen() → coloca o socket no modo “aguardando conexões”.
- accept() → aceita uma conexão vinda de um cliente.

Depois disso, cliente e servidor podem usar:

- send() / recv() (ou write() / read()) para trocar mensagens.

RPC síncrono

Exemplo prático:

Função remota **append**

```
new_list = append([1, 2, 3])
```

Operações de socket para TCP

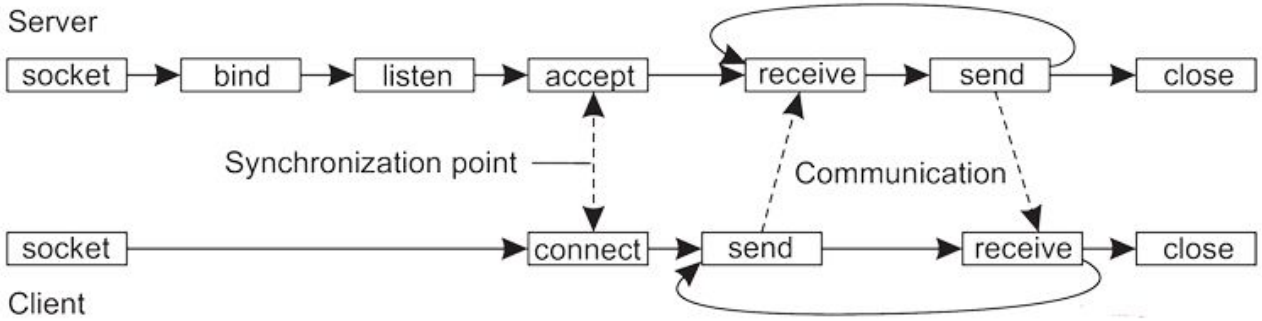


Figure 4.18: Connection-oriented communication pattern using sockets.

RPC síncrono

Exemplo prático:

Função remota **append**

```
new_list = append([1, 2, 3])
```

Exemplo de RPC com uso de Sockets

Abra o arquivo servidor_RPC.py e cliente_RPC.py

RPC síncrono

Exemplo prático:

Função remota `append`

```
new_list = append([1, 2, 3])
```

Exemplo de RPC com uso de Sockets - Vários clientes

Abra o arquivo servidor_lock_RPC.py e clientes_RPC.py

RCP - PASSAGEM DE PARÂMETROS

Exemplo prático:

Função remota **append**

```
new_list = append([1, 2, 3], [10, 20])
```

Vamos empacotar as informações utilizando pickle ou json

```
JSON:
{
  "operation": "append",
  "data": [1, 2, 3]
}
```

Saida: **string JSON**, que será enviada pela rede como bytes (.encode("utf-8")).

No servidor...

- o **servidor** recebe a sequência de bytes, e desserializa (json.loads(...)),
- e executa localmente.

```
JSON:
{
  "operation": "append",
  "data": [1, 2, 3]
}
```

Saida: **string JSON**, que será enviada pela rede como bytes (.encode("utf-8")).

No servidor...

- o **servidor** recebe a sequência de bytes, e desserializa (json.loads(...)),
- e executa localmente.

```
JSON:
{
  "operation": "append",
  "data": [1, 2, 3]
}
```

Alguns outros exemplo relacionados a metadados:
- JSON,
- XML,
- Protocol Buffers,
- Pickle (Python).

Ponteiros/referências entre processos

- Um **ponteiro** (ou referência) em C, Java, Python, só tem significado **dentro do espaço de endereçamento do processo que o criou**.

Exemplo: Considere que dbList aponta para o endereço de memória `0x7ffeab123` no cliente, no servidor **esse endereço não significa nada** (lá pode apontar para uma área completamente diferente, ou ser inválido).

Ponteiros/referências entre processos

1. Não permitir ponteiros/referências
2. Copiar por valor
3. Suporte da linguagem para serialização automática
4. Referências globais

Ponteiros/referências entre processos

1. Não permitir ponteiros/referências
   - altamente indesejável;
   - parâmetros de referência são frequentemente usados (vetores, matrizes, strings, objetos etc.);
   - podemos simplesmente copiar toda a estrutura de dados à qual o parâmetro se refere, substituindo o mecanismo de cópia por referência.

Ponteiros/referências entre processos

2. Copiar por valor

- Em vez de passar a referência, o *stub* copia o conteúdo da estrutura e envia.
- Se necessário, atualiza a cópia de volta no cliente.

Otimização: se os dados referenciados serão apenas lidos, não há necessidade de copiá-los de volta quando a chamada terminar.

Ponteiros/referências entre processos

3. Suporte da linguagem para serialização automática
   - tipos de dados complexos podem ser manipulados quando a linguagem de programação oferece suporte adequado.
     - Em linguagens como Python e Java, é possível definir classes personalizadas e aplicar mecanismos automáticos de serialização e desserialização.
   - estruturas de dados grandes, aninhadas ou complexas, a desserialização automática pode não estar disponível, ou ineficiente.

Ponteiros/referências entre processos

4. Referências globais

- se o cliente e o servidor tiverem acesso ao mesmo sistema de arquivos, em vez de mandar o ponteiro, manda-se um identificador válido para os dois.

  Exemplo: um **file handle** (descritor de arquivo)

- Nesse caso, cliente e servidor devem **combinar previamente** o que aquele identificador significa e quais operações podem ser feitas com ele.

RPC assíncrono (Asynchronous RPC)

Problema do RPC clássico:

- No RPC tradicional, **o cliente bloqueia até receber a resposta** do servidor.

- Isso pode ser ineficiente quando:
  1. Não há resultado a retornar.
  2. O cliente precisa contatar **vários servidores simultaneamente**.

RPC assíncrono (Asynchronous RPC)

Funcionamento:

1. O cliente envia a chamada RPC.
2. O servidor envia imediatamente uma confirmação de que vai processar a chamada.
3. O cliente **não bloqueia**, continua executando outras tarefas.
4. O servidor processa a chamada localmente.

RCP – VARIAÇÕES DE RPC

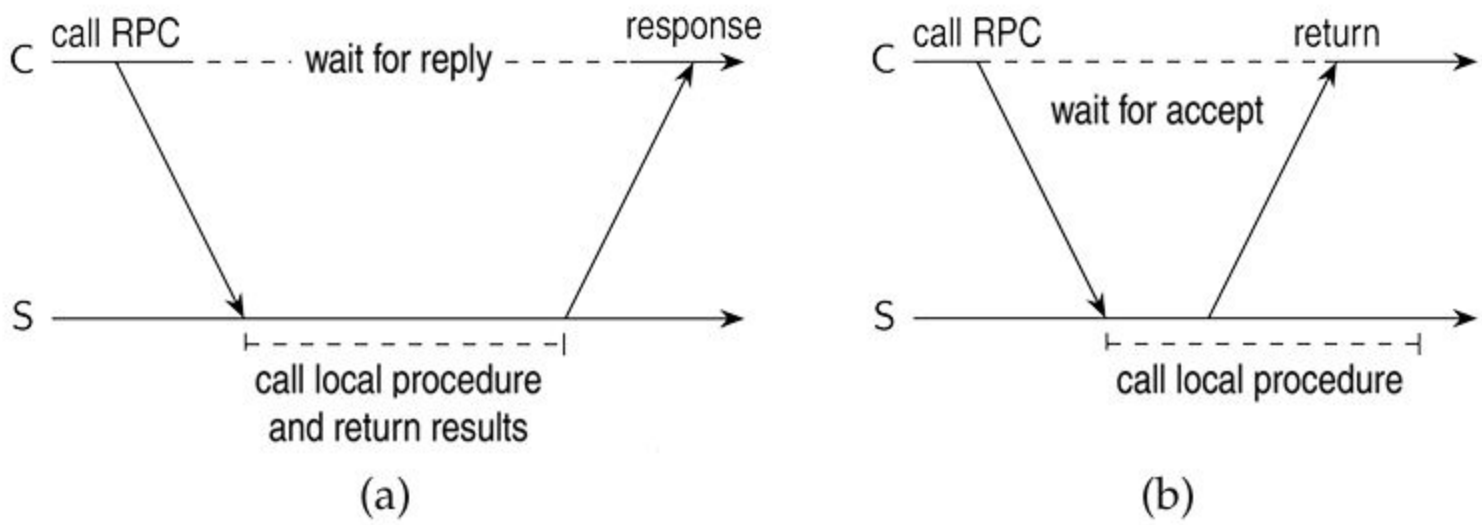


Figure 4.14: (a) The interaction between client and server in a traditional RPC. (b) The interaction using asynchronous RPC.

RPC síncrono adiado (*Deferred synchronous* RPC)

- É uma **variação que combina assincronia com *callback***.

  Passos:

  1. O cliente envia a chamada e espera **apenas pelo aceite** do servidor.
  2. Continua executando outras tarefas.
  3. Quando o servidor termina, envia a resposta.
  4. Uma função ***callback*** (retorno) no cliente é acionada automaticamente para processar o resultado.

RPC síncrono adiado (*Deferred synchronous* RPC)

- Uma forma de implementar envolve criar uma **thread separada** que fica **bloqueada** esperando o evento (mensagem do servidor).
- Quando o evento ocorre, a thread dispara a função ***callback***, sem interromper o fluxo principal do cliente.

RPC Multicast

Variação de RPC em que o **cliente envia uma única chamada remota**, mas essa chamada é **recebida por múltiplos servidores ao mesmo tempo**.

- Os servidores processam a requisição em **paralelo**.
- O cliente pode receber **uma ou várias respostas**, dependendo da aplicação.

RPC Multicast

Funcionamento

1. O **cliente faz um pedido** (ex: `append([1,2],dblist)`).
2. Esse pedido é enviado via **endereço multicast** (ou via vários sockets para servidores diferentes).
3. Todos os servidores que escutam esse grupo recebem a chamada.
4. Cada servidor **processa a requisição localmente**.
5. Cada servidor envia sua **resposta de volta ao cliente**.

RPC Multicast

Nesse método o cliente pode:

- esperar **todas as respostas**.
- esperar só a **primeira resposta válida**.
- esperar até que a **maioria** retorne o mesmo valor (útil em replicação tolerante a falhas).
- **mesclar resultados** (ex: dividir um trabalho entre servidores e juntar tudo).

RPC Multicast

Vantagens:

- **Tolerância a falhas**: ter servidores replicados (se 1 falhar, outro responde).
- **Distribuição de carga**: dividir dados e cada servidor processa uma parte.
- **Maior eficiência** em consultas paralelas (ex: vários bancos de dados em servidores diferentes).